

Q1:

1.4. Model Selection

Model with Highest Validation Classification Accuracy

Model: $\lambda_1 = 1.0, \lambda_2 = 1.0$

Validation Accuracy: 0.9022

Validation RMSE: 0.0501

Test Metrics:

Test Accuracy: 0.8928

Test RMSE: 0.0492

This model achieves the best classification performance, indicating that equal weighting of classification and regression losses ($\lambda_1 = \lambda_2 = 1$) gives a balanced representation learning that supports class discrimination.

Model with Lowest Validation Regression RMSE

Model: $\lambda_1 = 0.5, \lambda_2 = 2.0$

Validation Accuracy: 0.8728

Validation RMSE: 0.0406

Test Metrics:

Test Accuracy: 0.8684

Test RMSE: 0.0667

This configuration emphasizes regression more heavily (higher λ_2). It results in lower RMSE but slightly weaker classification accuracy, showing the trade-off between the two tasks.

Trade-off between λ_1 and λ_2

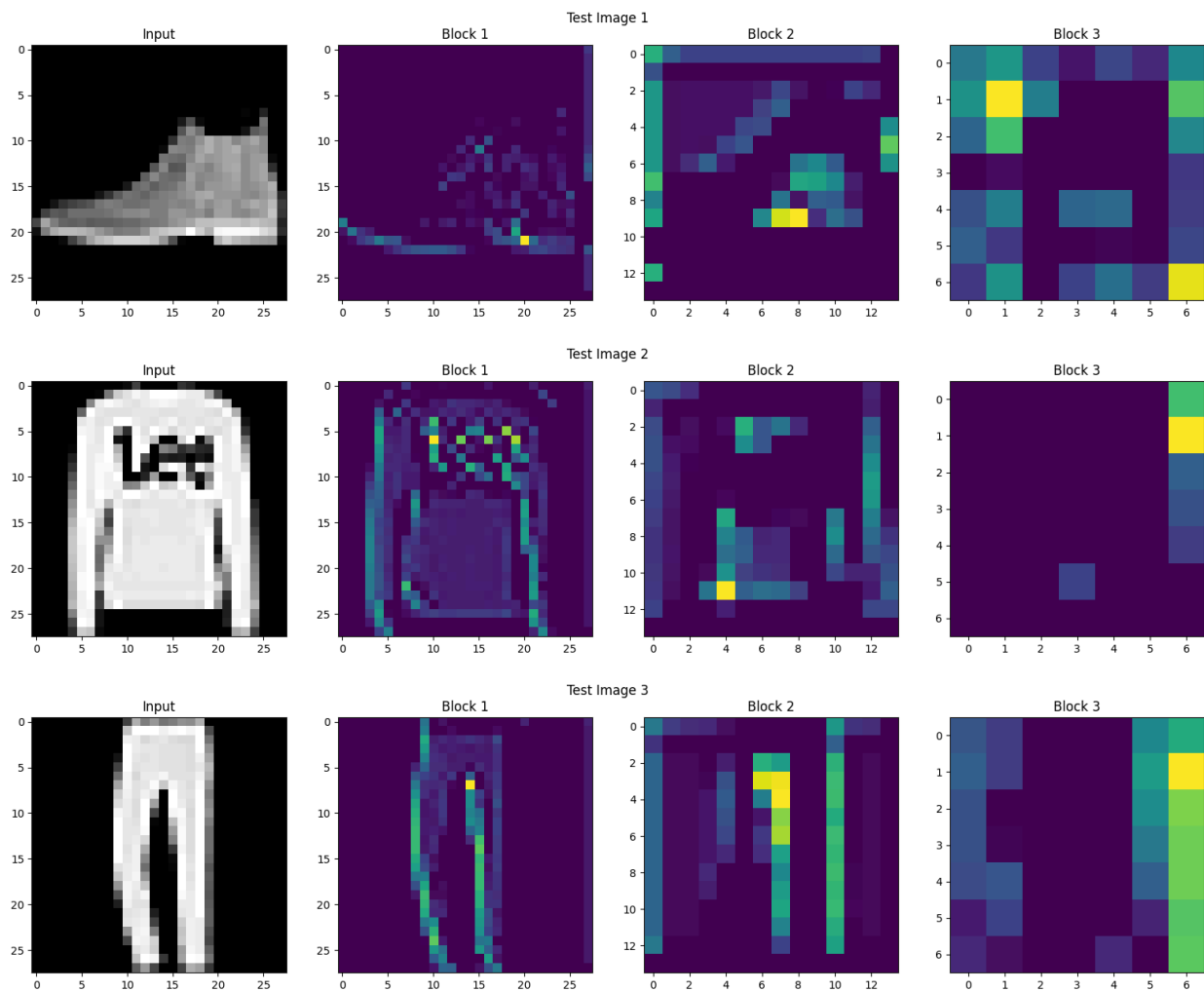
When λ_1 (classification weight) is higher, the network prioritizes learning discriminative features useful for class separation, improving accuracy but slightly hurting regression metrics.

When λ_2 (regression weight) is higher, the model focuses on learning continuous ink-related features, improving RMSE but reducing categorical precision.

A balanced configuration ($\lambda_1 = \lambda_2 = 1$) provides the best joint optimization, yielding high accuracy with stable regression error — making it the most practical choice for multitask training.

Feature Map Visualization

1. Intermediate Feature Maps



To understand how the model learns hierarchical features, the intermediate outputs after each convolutional block (Block 1, Block 2, Block 3) were extracted using hooks. The visualizations below show the activation maps for three representative test images: Shoe, Sweatshirt, and Trouser.

Test Image 1 – Shoe

Block 1: Captures edges, contours, and textures of the shoe, especially the sole and upper parts. Bright activations correspond to high-contrast edges and texture transitions.

Block 2: Highlights structured regions, such as the curved shoe outline and uniform body, showing the network learning localized patterns.

Block 3: Shows abstract, spatially compressed activations, representing the overall shoe shape and class-level information used for classification.

Test Image 2 – Sweatshirt

Block 1: Detects sharp vertical and horizontal edges along sleeves and collar, as well as text contours on the front

Block 2: Begins grouping these edges into mid-level shapes, like sleeve-body separation and text blocks.

Block 3: Compresses activations into broad semantic regions - capturing the overall garment shape rather than fine text, showing the model’s shift from pixel-level to concept-level representation.

Test Image 3 – Trouser

Block 1: Highlights parallel vertical edges, corresponding to trouser legs.

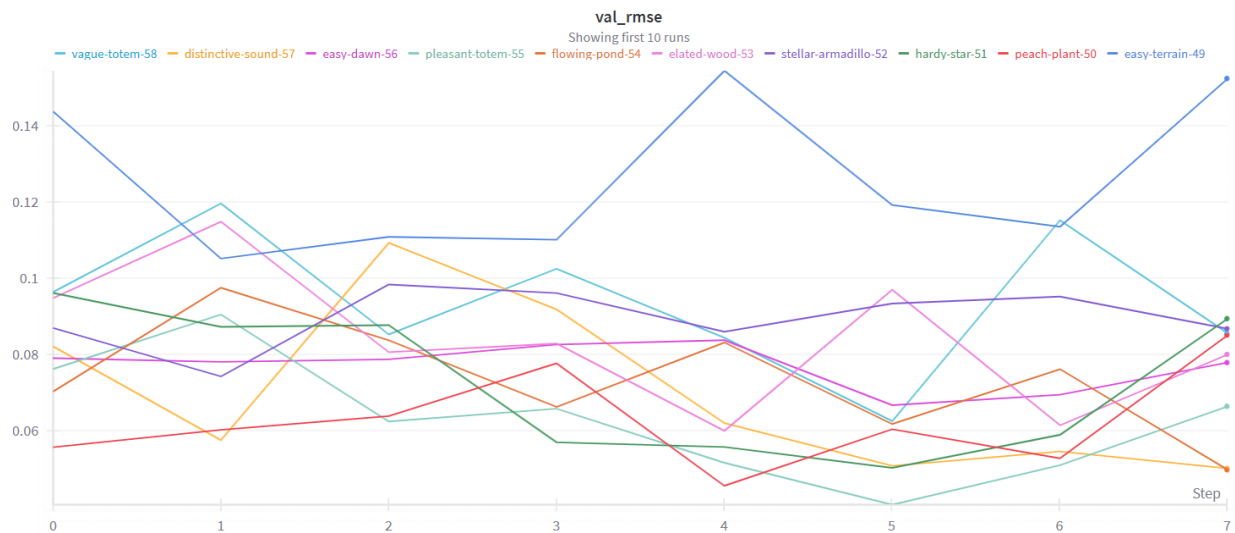
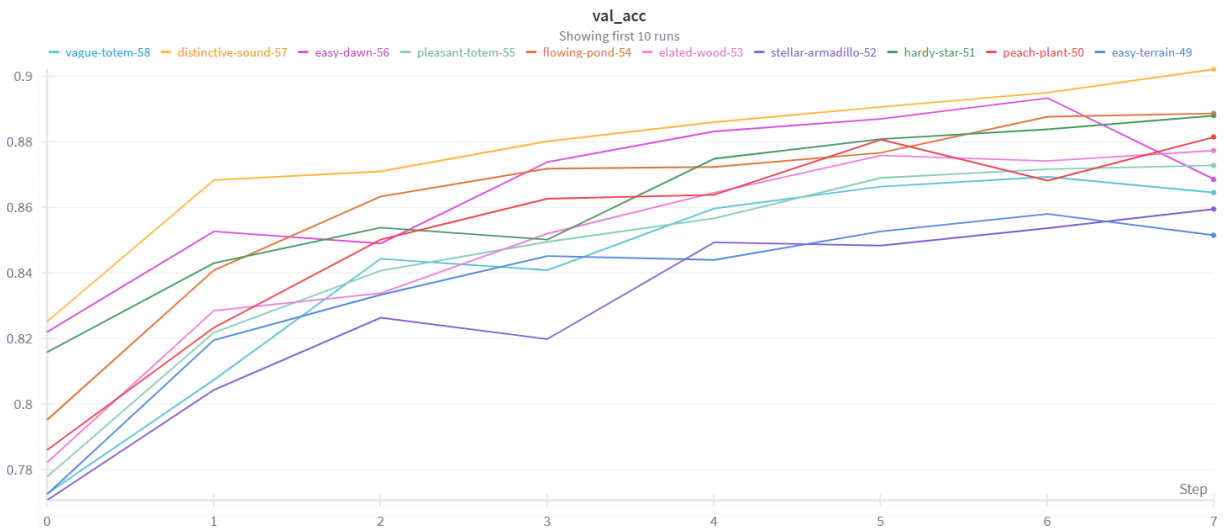
Block 2: Forms stripe-like texture groupings, reflecting fabric continuity and symmetry.

Block 3: Focuses on high-level segmentation, distinctly separating the two leg regions - crucial for both category recognition and continuous ink mapping.

2. Interpretation

Layer	Type of Features Learned	Contribution to Tasks
Block 1 (Low-level)	Basic edges, contours, intensity gradients	Helps detect shape outlines and fine details — essential for accurate ink boundary regression and object edges for classification.
Block 2 (Mid-level)	Local textures, repetitive patterns, regional contrasts	Learns fabric patterns, sleeve/sole/text distinctions — supports both ink density prediction and recognition of clothing types.
Block 3 (High-level)	Abstract shapes, semantic object parts	Encodes class-level structures (e.g., shoe vs trouser vs top), aiding classification, while maintaining smooth ink distribution understanding across object regions.

Wandb dashboard: https://wandb.ai/yogeswari05-iiit-hyderabad/multi_task_fashion



Best Models:

	Model Configuration	Validation Accuracy	Validation RMSE	Test Accuracy	Test RMSE
Best Classification Model	$\lambda_1=1.0$, $\lambda_2=1.0$, lr=0.001, dropout=0.2, filters=32, batch=128, AdamW	0.9022	0.0501	0.8928	0.0492
Best Regression Model	$\lambda_1=0.5$, $\lambda_2=2.0$, lr=0.001, dropout=0.4, filters=32, batch=64, Adam	0.8728	0.0406	0.8684	0.0667

Approach:

The task was a multi-task learning problem combining:

Classification: Predicting the clothing category (e.g., shoe, trouser, top).

Regression: Predicting the ink intensity or density associated with each pixel/region.

To jointly optimize both objectives, a shared convolutional feature extractor was used with two output heads:

A classification head using a softmax layer with cross-entropy loss, weighted by λ_1 .

A regression head using mean squared error (MSE) loss, weighted by λ_2 .

The **total loss**:

$$L_{\text{total}} = \lambda_1 \cdot L_{\text{class}} + \lambda_2 \cdot L_{\text{reg}}$$

Varying λ_1 and λ_2 allowed analysis of the trade-off between discrete class accuracy and continuous ink prediction accuracy.

Model Architecture

Input: 28×28 grayscale clothing images.

Convolutional Blocks: Three convolutional blocks, each with Conv2D → BatchNorm → ReLU → MaxPool layers.

Classification Head: Fully connected layer → softmax output.

Regression Head: Fully connected layer → single linear output neuron.

This design allows low-level shared feature extraction and separate task-specific learning at the final layers.

Hyperparameters

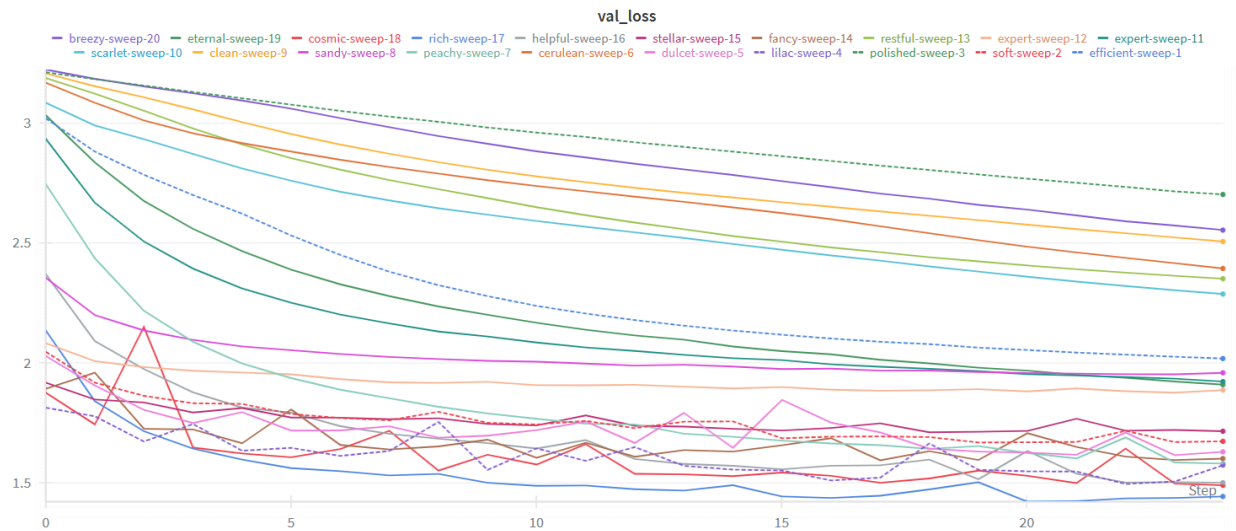
Hyperparameter	Values Tried
Learning Rate (lr)	{0.0005, 0.001}
Dropout	{0.2, 0.4}
Number of Conv Blocks	3

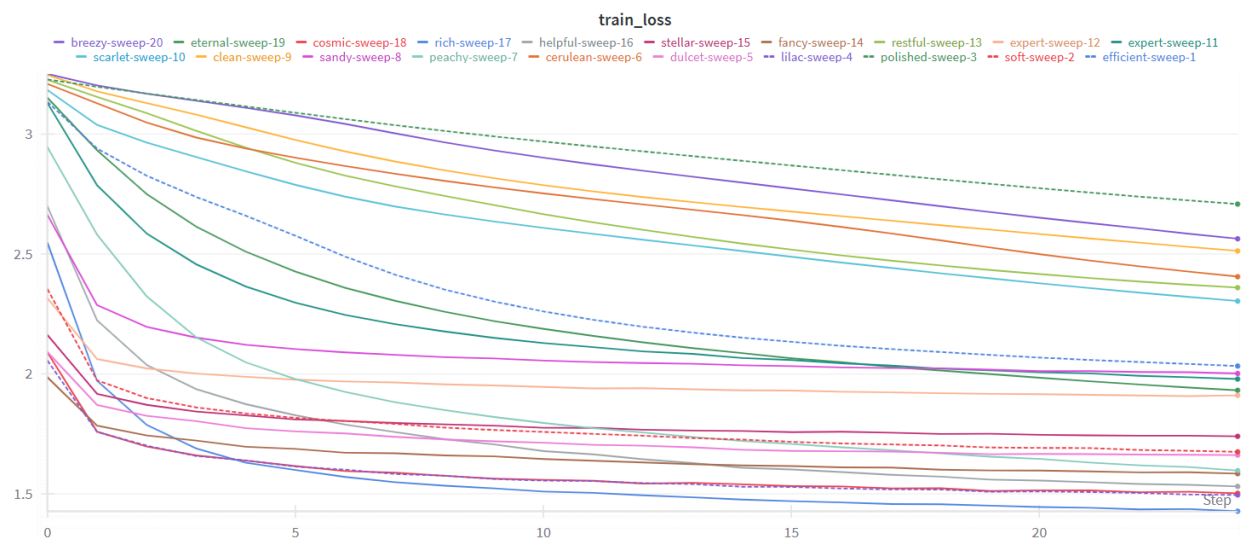
Number of Filters	{16, 32}
Batch Size	{64, 128}
Optimizer	Adam / AdamW
λ_1 (classification weight)	{0.5, 1.0, 2.0}
λ_2 (regression weight)	{0.5, 1.0, 2.0}

Q2:

Wandb dashboard link: <https://wandb.ai/yogeswari05-iiit-hyderabad/ image-colourisation-cifar10>
<https://wandb.ai/yogeswari05-iiit-hyderabad/ image-colourisation-cifar10/sweeps/0hdeee43?nwuser=yogeswari05>

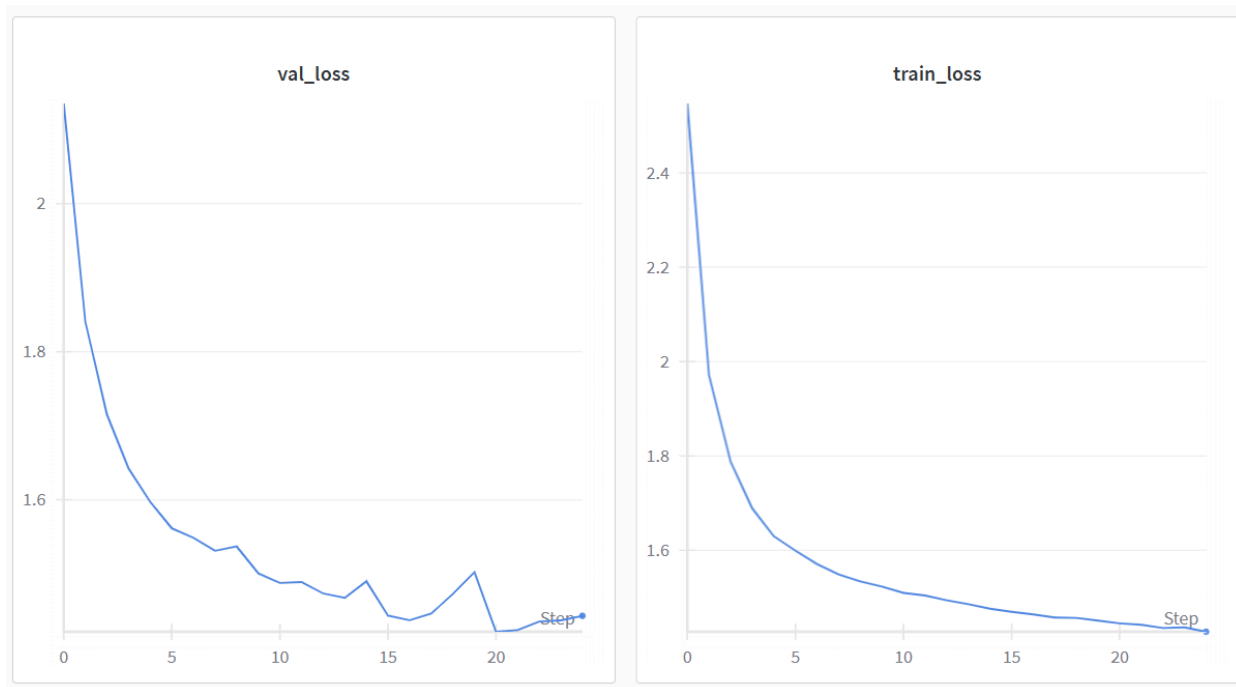
Training and validation loss curves:





Analytical Questions: Refer to [README.md](#) file.

Best Run:



Hyperparameters:

Batch size: 128

Kernel size: 5

Learning rate (lr): 0.0003

Number of filters (NF): 32

Optimizer: Adam

Epochs: 25

Best Validation Loss: 1.4216

Training Loss: 1.4274

Validation Loss: 1.4431

Observation:

The best run was achieved using the Adam optimizer with a learning rate of 0.0003 and kernel size of 5. The model converged well, reaching a minimum validation loss of 1.4216, indicating stable generalization performance across training and validation sets.

Q3:

Is there any internal node that has two child leaf nodes corresponding to the same sentiment class? Why would having two children predict the same class make sense?

Yes, there are internal nodes whose two children predict the same sentiment class. Because the split variable at that node does not significantly separate the classes. Both subsets may still be dominated by the same majority label even after the split. This can occur before pruning, as the decision tree algorithm splits to increase purity, not necessarily to change class labels. Even though two leaves predict the same class, the split might still yield a small impurity improvement, justifying its presence. This makes sense because the decision tree algorithm splits to improve purity, not necessarily to change the class label.

What are the values of max_depth and min_samples_leaf for the best tree found by grid search?

max_depth = 32

min_samples_leaf = 3

Top 10 Important Features	Importance	10 Random Low-Importance Features	Importance
return	0.032990	quality_is	0.0
excel	0.029485	better_off	0.0
great	0.028984	this_printer	0.0
worst	0.028407	worth_every	0.0
poor	0.026748	listen	0.0
disappoint	0.024952	make_up	0.0

your_money	0.018002	it_down	0.0
i_love	0.017955	harsh	0.0
the_best	0.017662	me_wrong	0.0
don't	0.017654	insult	0.0

Number of near-zero importance terms ($< 1e-5$): 7,296 features

What is the value of max_features of your best forest?

max_features=33

What's the maximum possible value of max_features?

7729 (the total vocabulary size).

Why is it beneficial to tune this hyper parameter?

max_features controls how many features are considered when splitting each node. Smaller max_features → more randomization, trees are less correlated → better generalization but possibly less accuracy on training data. Larger max_features → trees become more similar → lower bias but higher variance. Hence, tuning it helps balance bias–variance tradeoff for best performance.

When fitting random forests, what is the primary tradeoff controlled by the n_estimators hyperparameter?

n_estimators = number of trees in the forest.

Tradeoff:

Small n_estimators: higher variance, unstable model.

Large n_estimators: more stable, lower variance, but higher computational cost.

Can you overfit by setting it to be too large?

Generally, no. Increasing n_estimators doesn't cause overfitting because each tree is trained on random subsets (bootstrapping + feature sampling). After a certain point, adding more trees only increases computation without improving accuracy.

Summarizing conclusions:

Model Type	n_trees	max_depth	Train Acc	Valid Acc	Test Acc
Simple DT	1	3	0.6458	0.6446	0.6458
Best DT	1	32	0.8772	0.7324	0.7489
Simple RF	100	3	0.8191	0.7972	0.7782
Best RF	100	32	0.9644	0.8512	0.8369

Increasing tree depth in DT improves training accuracy but risks overfitting (seen in Best DT).

Random Forests significantly outperform Decision Trees on validation/test data due to ensemble averaging reducing variance.

The best RF (depth=32, max_features=33) achieves the highest balanced accuracy (≈ 0.85 on validation, ≈ 0.84 on test).

This shows that ensemble methods like Random Forests provide a better bias–variance tradeoff than a single deep tree.

max_features controls diversity between trees: smaller values $\rightarrow$ more diversity $\rightarrow$ lower variance.